

[Open in app](#)[Sign up](#)[Sign in](#)**Medium** SearchGeek Culture · [Follow publication](#)

Raspberry Pi: C++ Libraries for working with I2C, SPI and UART

Sebastian · [Follow](#)

Published in Geek Culture

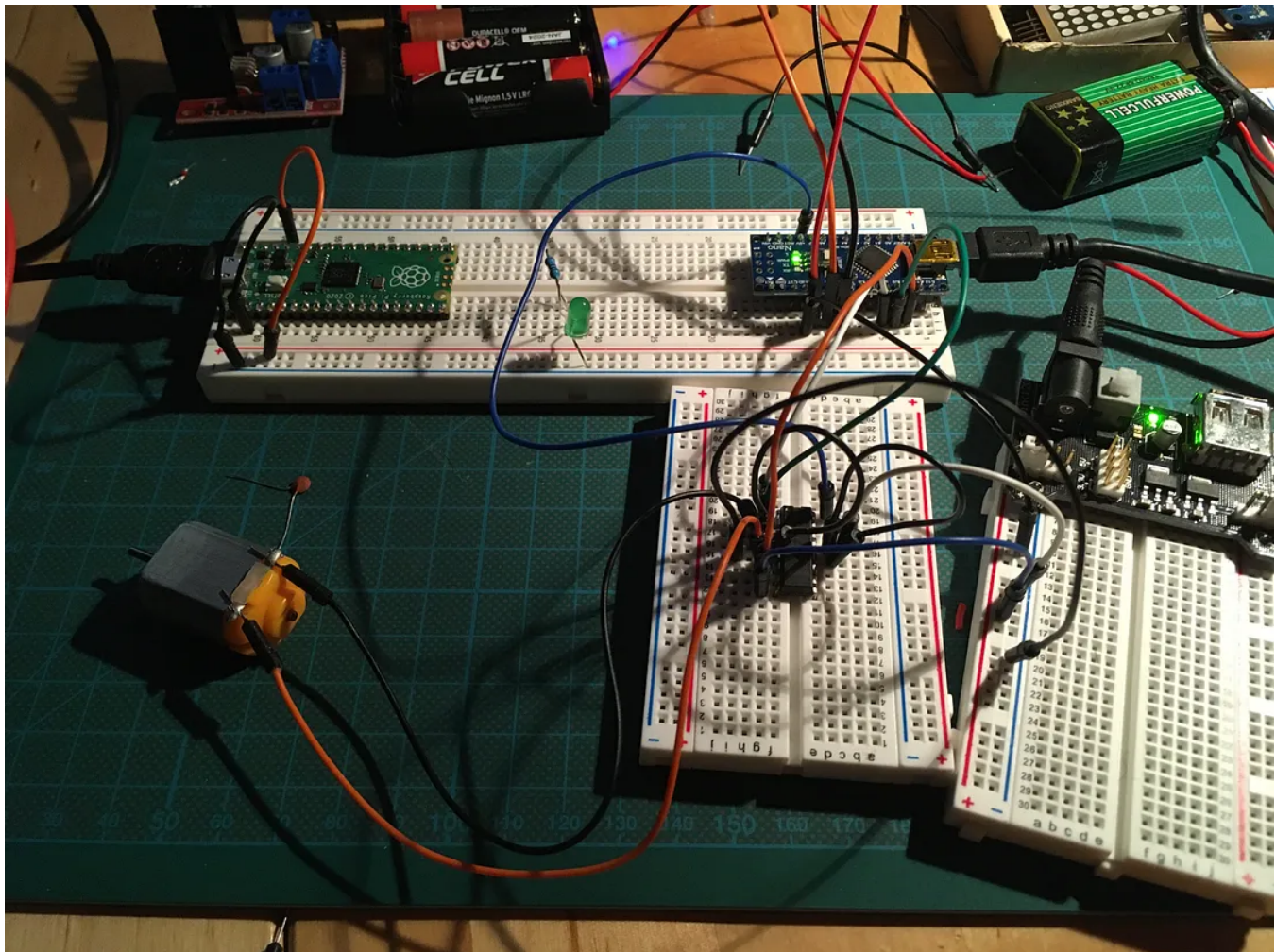
5 min read · Aug 22, 2021



Listen



Share



The Raspberry Pi is a single board computer with now 4 revisions and a minimalistic zero variant. It is a popular choice for different projects because of its small size, efficient power consumption, processing speed and by being a full Linux based computer.

One way to connect multiple single-board computers and/or microcontrollers is direct wiring. For this purpose, the most common used protocols are I2C, SPI and UART. Preceding articles in the blog series explained the principles of these protocols, and presented specific C-libraries for the Arduino. In this article, I will explain C++ libraries that enable to work with these protocols on a Raspberry Pi. For each protocol, I researched useable libraries, and give a short explanation and code example. Please not that the examples are not developed by me, but come from the libraries documentation, and should serve as a basis for a concrete working example.

This article originally appeared at [my blog adamantium.com](http://myblog.admantium.com).

I2C

I2C can be supported with the help of the SMBus protocol, which is described as a specific variant of the I2C bus. This protocol is available as Linux Kernel module. To use it, you need to configure your Raspberry Pi. In a terminal, run `raspi-config`, select 3 Interfacing Options and P5 I2C .

Following the code example from kernel.org, you will need to open the device file that represents the connected I2C device, and then send SMBus commands by writing to the devices registers.

```
// SOURCE: https://www.kernel.org/doc/Documentation/i2c/dev-  
interface */  
#include <linux/i2c-dev.h>  
#include <i2c/smbus.h>  
  
int file;  
int adapter_nr = 2;  
char filename[20];  
  
snprintf(filename, 19, "/dev/i2c-%d", adapter_nr);
```

```
file = open(filename, O_RDWR);
if (file < 0) {
    exit(1);
}

int addr = 0x40;

if (ioctl(file, I2C_SLAVE, addr) < 0) {
    exit(1);
}

__u8 reg = 0x10;
__s32 res;
char buf[10];

res = i2c_smbus_read_word_data(file, reg);
if (res < 0) {
    /* ERROR HANDLING: i2c transaction failed */
} else {
    /* res contains the read word */
}

buf[0] = reg;
buf[1] = 0x43;
buf[2] = 0x65;
if (write(file, buf, 3) != 3) {
    /* ERROR HANDLING: i2c transaction failed */
}
```

SPI

To work with the SPI, you also need to add a specific Kernel module: Spidev. The Raspberry Pi supports this module, you need to configure it by invoking `raspi-config`, and then select 3 Interfacing Options and P4 SPI.

To access SPI functions with C/C++, you can use the spidev wrapper library. Following the example code, you need to configure the SPI connection, then open the device that you want to connect to, and then use the library method for reading and writing data.

```
//SOURCE: https://raw.githubusercontent.com/milekium/spidev-lib/
master/sample/spidev-testcpp.cc
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <spidev_lib++.h>
```

```

spi_config_t spi_config;
uint8_t tx_buffer[32];
uint8_t rx_buffer[32];

int main( void)
{
    SPI *mySPI = NULL;

    spi_config.mode=0;
    spi_config.speed=1000000;
    spi_config.delay=0;
    spi_config.bits_per_word=8;

    mySPI=new SPI("/dev/spidev1.0",&spi_config);

    if (mySPI->begin())
    {
        memset(tx_buffer,0,32);
        memset(rx_buffer,0,32);
        sprintf((char*)tx_buffer,"hello world");
        printf("sending %s, to spidev2.0 in full duplex \n ",
(char*)tx_buffer);
        mySPI->
>xfer(tx_buffer,strlen((char*)tx_buffer),rx_buffer,strlen((char*)tx_
buffer));
        printf("rx_buffer=%s\n",(char *)rx_buffer);
        //mySPI->end();
        delete mySPI;
    }
    return 1;
}

```

UART

An UART connection can be established with common C libraries. Following the very detailed explanation in [this article](#), you will need to open the device, then define various properties of the TTY device by using a `termios struct`, and then write to and read from the serial port.

```

// SOURCE: https://blog.mbedded.ninja/programming/operating-systems/
linux/linux-serial-ports-using-c-cpp/
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <errno.h>
#include <termios.h>

```

```
#include <unistd.h>

int main() {
    int serial_port = open("/dev/ttyUSB0", O_RDWR);

    struct termios tty;

    if(tcgetattr(serial_port, &tty) != 0) {
        printf("Error %i from tcgetattr: %s\n", errno,
strerror(errno));
        return 1;
    }

    tty.c_cflag &= ~PARENB;
    tty.c_cflag &= ~CSTOPB;
    tty.c_cflag &= ~CSIZE;
    tty.c_cflag |= CS8;
    tty.c_cflag &= ~CRTSCTS;
    tty.c_cflag |= CREAD | CLOCAL;

    tty.c_lflag &= ~ICANON;
    tty.c_lflag &= ~ECHO;
    tty.c_lflag &= ~ECHOE;
    tty.c_lflag &= ~ECHONL;
    tty.c_lflag &= ~ISIG;
    tty.c_iflag &= ~(IXON | IXOFF | IXANY);
    tty.c_iflag &= ~(IGNBRK|BRKINT|PARMRK|ISTRIP|INLCR|IGNCR|ICRNL);

    tty.c_oflag &= ~OPOST;
    tty.c_oflag &= ~ONLCR;

    tty.c_cc[VTIME] = 10;
    tty.c_cc[VMIN] = 0;

    cfsetispeed(&tty, B9600);
    cfsetospeed(&tty, B9600);

    if (tcsetattr(serial_port, TCSANOW, &tty) != 0) {
        printf("Error %i from tcsetattr: %s\n", errno,
strerror(errno));
        return 1;
    }

    unsigned char msg[] = { 'H', 'e', 'l', 'l', 'o', '\r' };
    write(serial_port, "Hello, world!", sizeof(msg));

    char read_buf [256];

    memset(&read_buf, '\0', sizeof(read_buf));

    int num_bytes = read(serial_port, &read_buf, sizeof(read_buf));

    if (num_bytes < 0) {
        printf("Error reading: %s", strerror(errno));
        return 1;
    }
}
```

```
}

printf("Read %i bytes. Received message: %s", num_bytes,
read_buf);

close(serial_port)
return 0;
```

Universal GPIO Access

The library libgpiod provides universal access to GPIOs of any device running Linux. It detects available GPIO, can read and write data to them, and wait for events to be triggered. With this, you can write ow code to talk UART to any connected device.

To install it, run the following commands:

```
apt-get install autoconf autoconf-archive libtool libkmod-dev pkg-
config
git clone https://github.com/brgl/libgpiod.git

cd libgpiod
./autogen.sh --enable-tools=yes --prefix=/usr/local/bin
make
sudo make install
```

If compilation and installation is successful, in the subfolder `./tools` you will find binaries like `gpiodetect` and `gpioinfo` that you can use to explore the GPIOs. See the following examples.

```
$> ./tools/gpiodetect
gpiochip0 [pinctrl-bcm2711] (58 lines)
gpiochip1 [raspberrypi-exp-gpio] (8 lines)

./tools/gpioinfo
gpiochip0 - 58 lines:
line  0:      "ID_SDA"          unused   input   active-high
line  1:      "ID_SCL"          unused   input   active-high
line  2:      "SDA1"            unused   input   active-high
line  3:      "SCL1"            unused   input   active-high
line  4:  "GPIO_GCLK"            unused   input   active-high
line  5:      "GPIO5"            unused   input   active-high
```

...

If you want to work with the library, read [this article](#) for a detailed introduction.

Conclusion

For working with I2C, SPI and UART on the Raspberry Pi, not only Python, but C++ libraries can be used as well. Specifically, you need to activate the I2C and SPI functions via `raspi-config`, which internally loads the appropriate Kernel modules. Then you choose a client library and other necessary C++ headers. Working with the libraries follows the same principles: Determine the connected device file, configure a connection object, open the device file, then read from/write to it. Finally, the handy library `libgpiod` can help you to access all GPIO pins directly, which can be helpful for debugging.

Raspberry Pi

Cpp

I2c

Spi

Uart



Follow

Published in Geek Culture

33K Followers · Last published Aug 31, 2023

A new tech publication by Start it up (<https://medium.com/swlh>).

Follow

Written by Sebastian

1.7K Followers · 97 Following

IT Project Manager & Developer

Responses (1)



Write a response

What are your thoughts?



Artem MELNYK

Sep 2, 2023



Dear Sebastian, have you ever tested the example for I2C ?



Reply

More from Sebastian and Geek Culture





Sebastian

Ansible: Working with Variables and Hostvars

When I started to write about my `infrastructure@home` project, Ansible became the central configuration management tool. In order to be...

Sep 13, 2020 🖱 40



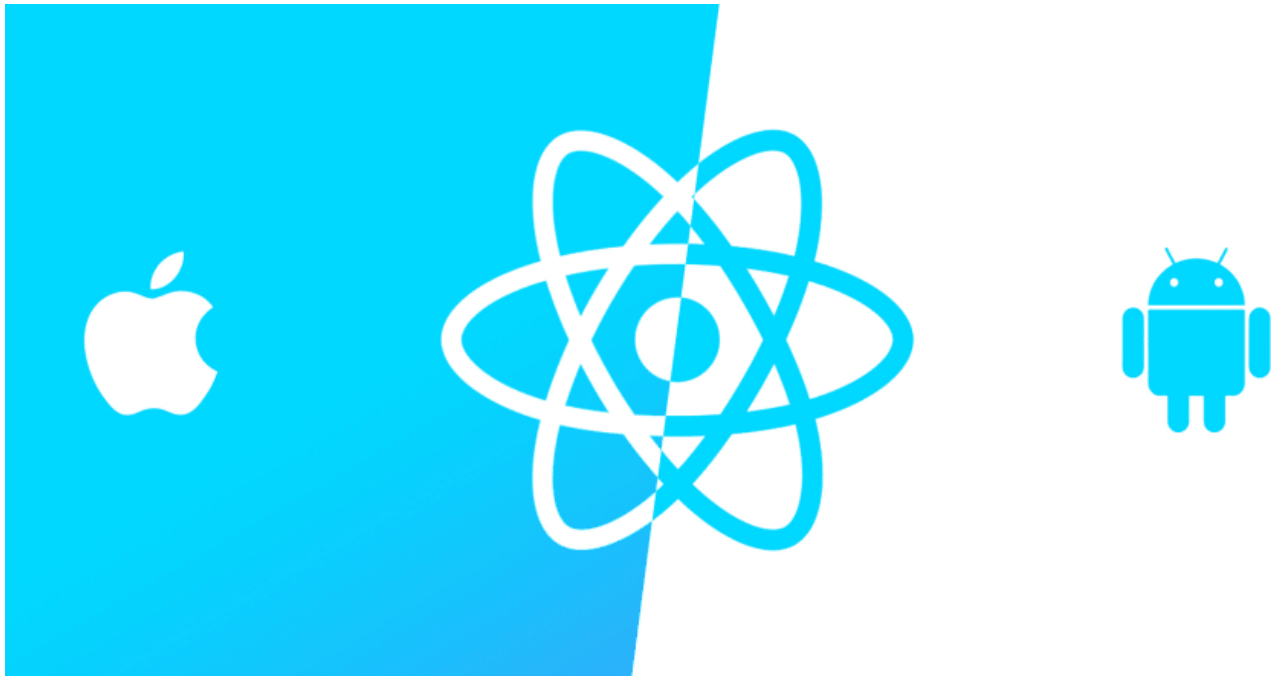
In Geek Culture by Hasitha Subhashana

Circuit Breaker Pattern (Design Patterns for Microservices)

In a distributed system we have no idea how other components would fail. Network issues could occur, components could fail or a router or a...

Jun 12, 2021 🖱 907 💬 11





In Geek Culture by Anshul Borawake

React Native Generate APK—Debug and Release APK

Generate Debug and Release APK in React Native; Windows, iOS and Linux

Apr 3, 2021 🖱️ 1.8K 💬 16



```
.program blink
```

```
set pindirs, 1
```

```
loop:
```

```
    set pins, 1 [31]
```

```
    set pins, 0 [31]
```



In Geek Culture by Sebastian

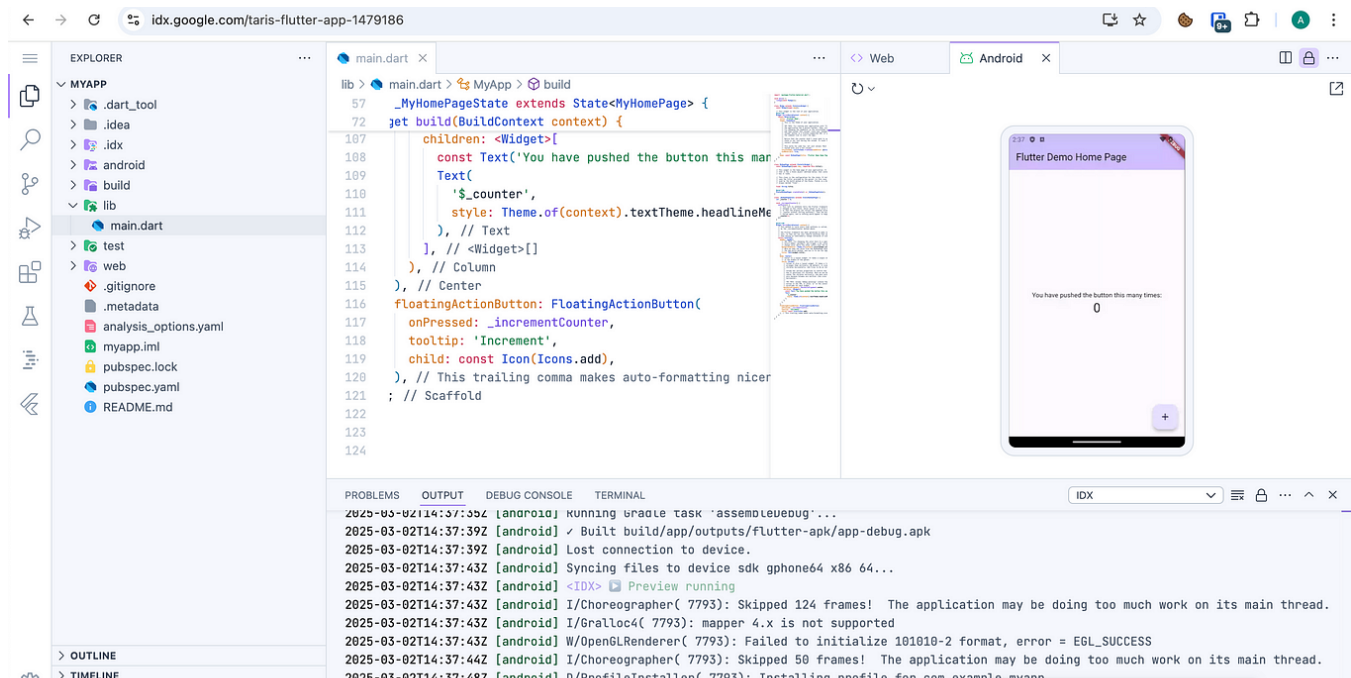
Raspberry Pico: Programming with PIO State Machines

In Microcontroller programming, interfacing with other hardware can be either very simple or very challenging. If the other hardware, e.g...

Jun 20, 2021 🖱 155 💬 3

[See all from Sebastian](#)[See all from Geek Culture](#)

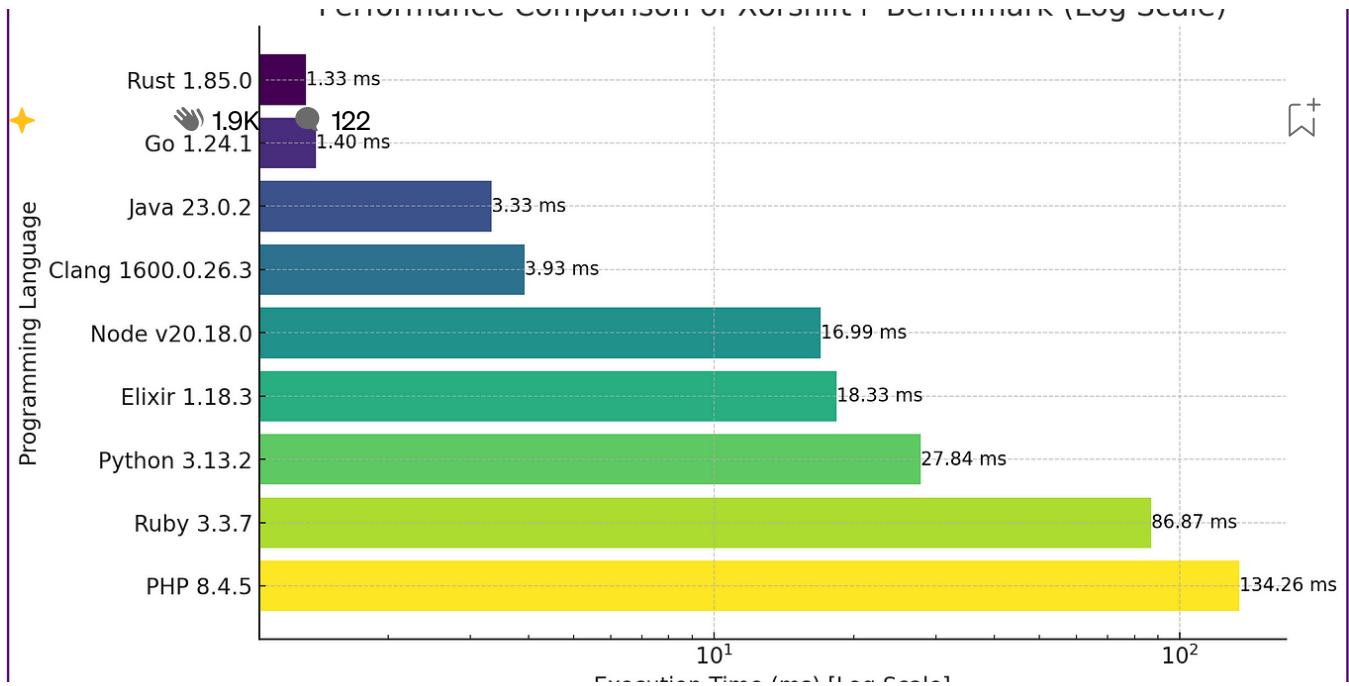
Recommended from Medium





In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer



In Effective Programmer by Naveed Khan

So, How Fast is Rust Anyway

Well, its pretty darn fast



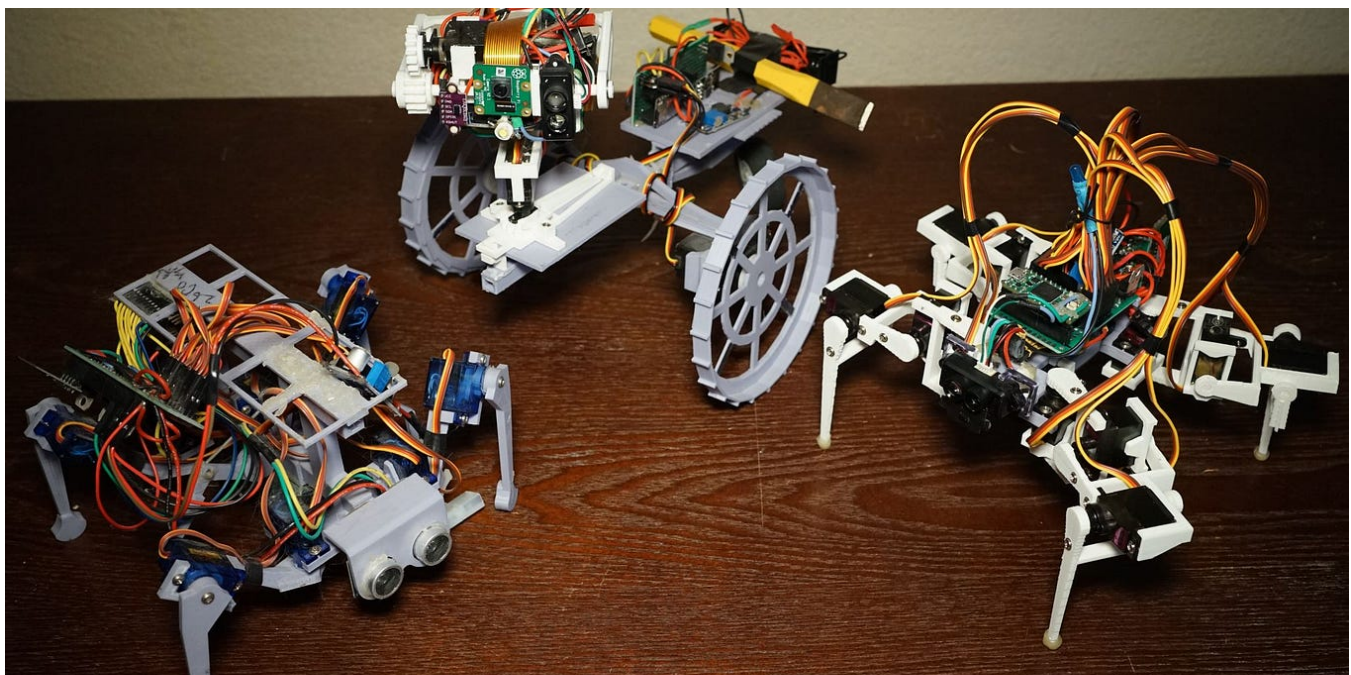
5d ago



301



21





Jacob David Cunningham

Basics on hobby robotics

This a generic overview of making basic hobby robotics with common off the shelf parts. This is not a comprehensive guide but a collection...

Apr 20, 2023 🖱 208 💬 4



Jan Kammerath

Fired “Kill Switch” Programmer Faces 10 Years In Jail: What Went Wrong?

The 55 year old Texan Davis Lu was just sentenced to 10 years in jail for pulling a “kill switch” on his employer EATON Corporation where...

★ 6d ago 🖱 1K 💬 56



Book (Optional)

Perceptual intelligence by Brian Wachler

Start Page

Start Reading Session

Total Pages Read

143

All-time pages read

Reading Sessions

4

Completed sessions

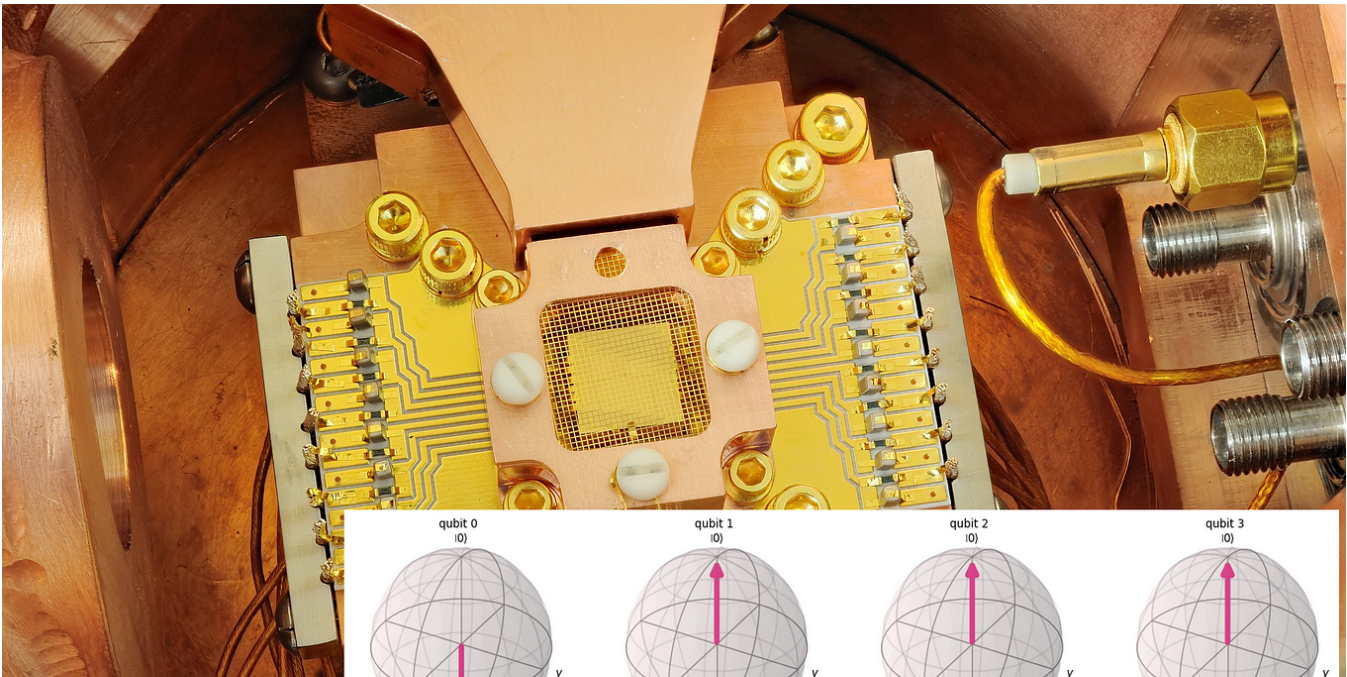
 Chris Dunlop

Is Cursor better than VS Code with Copilot? Absolutely and it's not close

Cursor 3.7 + Claude is the first time for me in coding that AI has got to a level that makes it absolutely ridiculous. It is far better...

Mar 8 452 11







In Level Up Coding by Dmitrii Eliuseev

Making a “Hello World” Program For a Quantum Computer

Let's Do Some Weird Stuff



4d ago



627



7



See more recommendations